BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: William Natcharian

ID: WRN25

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `~/Desktop/4750 Cloned Repositories/hw4-wrn25_hw4`
```

```
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

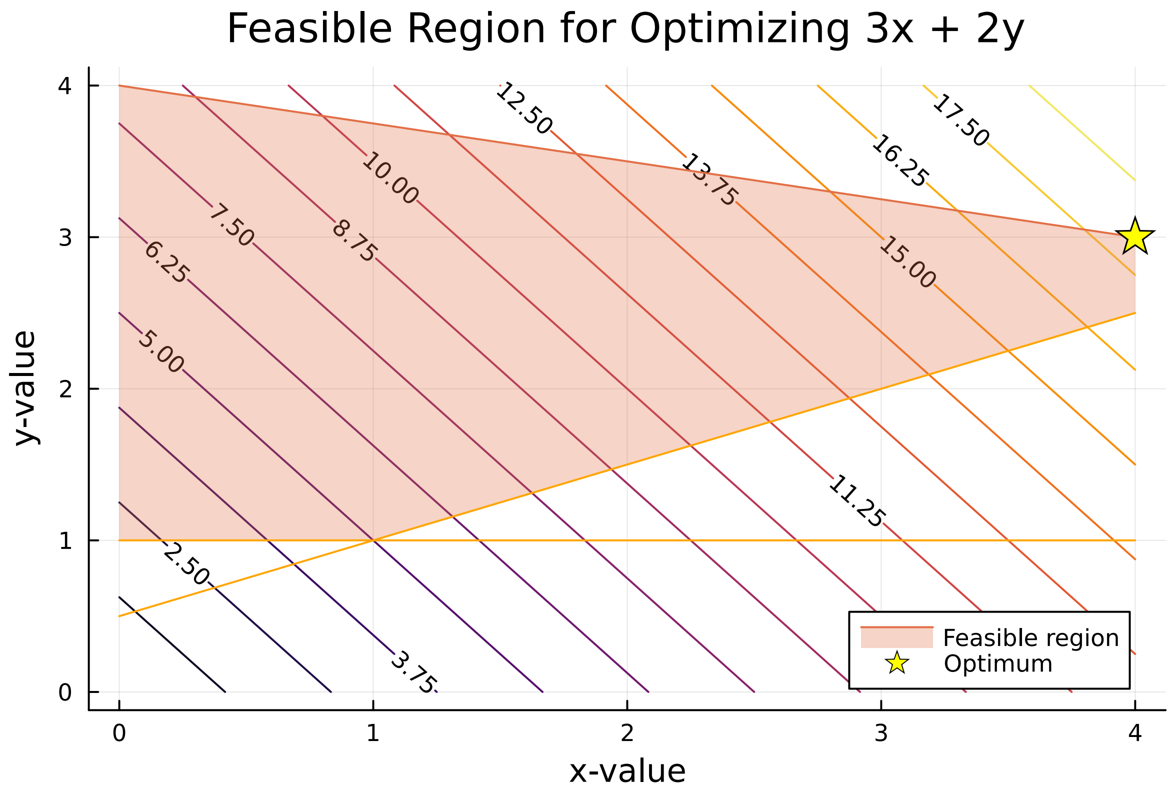
Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

–>

```
x = 0:0.01:4

# Example: y <= 1 and y <= 3 - 2x/3
#uppery = max.(1.0, 3 .- 2 .* x ./ 3)
#uppery = max.(10 .+x*0, 10 .- x)
# Example: y <= 5/3 + x
#lowery = 5 .+ x

lowery = min.(4, 4 .- x*0.25)
uppery = max.(1, 0.5 .+ x*0.5)

line = 0.5 .+ x*0.5
extra = 1:0.01:4

f1(x, y) = 3x + 2y
optimum = f1(4,3)
x = 0:0.01:4
y = 0:0.01:4
```

```
p1 = contour(x, y, f1, title="Feasible Region for Optimizing 3x + 2y", xlabel="x-value", ylal
plot!(p1, x, lowery, fillrange = uppery, fillalpha = 0.3, label = "Feasible region")
plot!(p1, x, line, color =:orange, label = "")
plot!(p1, x, 1 .+ x*0, color =:orange, label = "")
scatter!(p1, [4], [3], label = "Optimum", markershape =:star5, markersize = 10, markercolor =
display(p1)
print("Optimum solution at (4,3) with a maximum value of $optimum")
```




```
Optimum solution at (4,3) with a maximum value of 18
```

–> The plot above shows the contour lines for the equation we are trying to maximize, 3x + 2y, with the feasible region plotted on top of it based on the four restricting equations.

–> The solution will always be found on one of the four corners of the feasible region. In this case it is obvious from the values of the contour lines that the optimum solution is in the top right corner. This optimum solution is found at (4,3) and has a value of 18.

–> Code Explanation: I used the contour() plotting function to add the obective vallue lines behind the feasible region. To create the feasible region I used the fillrange() function within the plotting function and defined the tops and bottoms by the corresponding minimums and maximum inequalities describing the region.

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Decision Variables:

- Space allocated to Soybeans in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{S0}, S_{S1}, S_{S2}
- Space allocated to Wheat in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{W0}, S_{W1}, S_{W2}
- Space allocated to Corn in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{C0}, S_{C1}, S_{C2}

Objective Function(s): To Maximize Profit per year

- Max $(S_{S0} * (2900 * 0.36 - 350 - 70 * 0) + S_{S1} * (3800 * 0.36 - 350 - 70 * 1) + S_{S2} * (4400 * 0.36 - 350 - 70 * 2) + S_{W0} * (3500 * 0.27 - 280 - 70 * 0) + S_{W1} * (4100 * 0.27 - 280 - 70 * 1) + S_{W2} * (4200 * 0.27 - 280 - 70 * 2) + S_{C0} * (5900 * 0.22 - 390 - 70 * 0) + S_{C1} * (6700 * 0.22 - 390 - 70 * 1) + S_{C2} * (7900 * 0.22 - 390 - 70 * 2))$
- –>
- Max $(694S_{S0} + 948S_{S1} + 1094S_{S2} + 555S_{W0} + 757S_{W1} + 714S_{W2} + 908S_{C0} + 1014S_{C1} + 1208S_{C2})$

Constraints:

- Average pesticide applications:
- $-0.8S_{S0} + 0.2S_{S1} + 1.2S_{S2} \leq 0$
- $-0.7S_{W0} + 0.3S_{W1} + 1.3S_{W2} \leq 0$
- $-0.6S_{C0} + 0.4S_{C1} + 1.4S_{C2} \leq 0$
- Total hectares used:
- $S_{S0} + S_{S1} + S_{S2} + S_{W0} + S_{W1} + S_{W2} + S_{C0} + S_{C1} + S_{C2} \leq 130$
- Total amount in kg produced:
- $2900S_{S0} + 3800S_{S1} + 4400S_{S2} \leq 250000$
- $3500S_{W0} + 4100S_{W1} + 4200S_{W2} \leq 250000$
- $5900S_{C0} + 6700S_{C1} + 7900S_{C2} \leq 250000$

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

```julia
plant_model = Model(HiGHS.Optimizer) # initialize model object
@variable(plant_model, S[1:9] >= 0) # non-negativity constraints
# uncomment the following lines and add the objective and constraints as needed for the model
@objective(plant_model, Max, 694S[1] + 948S[2] + 1094S[3] + 555S[4] + 757S[5] + 714S[6] + 908

@constraint(plant_model, soy_average, -0.8S[1] + 0.2S[2] + 1.2S[3]  <= 0)
@constraint(plant_model, wheat_average, -0.7S[4] + 0.3S[5] + 1.3S[6]  <= 0)
@constraint(plant_model, corn_average, -0.6S[7] + 0.4S[8] + 1.4S[9]  <= 0)

@constraint(plant_model, total_hectares, S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8

@constraint(plant_model, total_kilos_soy, 2900S[1] + 3800S[2] + 4400S[3] <= 250000)
@constraint(plant_model, total_kilos_wheat, 3500S[4] + 4100S[5] + 4200S[6] <= 250000)
@constraint(plant_model, total_kilos_corn, 5900S[7] + 6700S[8] + 7900S[9] <= 250000)

print(plant_model) # this outputs a nicely formatted summary of the model so you can check yo

optimize!(plant_model)
```

```
Max 694 S[1] + 948 S[2] + 1094 S[3] + 555 S[4] + 757 S[5] + 714 S[6] + 908 S[7] + 1014 S[8] +
Subject to
 soy_average : -0.8 S[1] + 0.2 S[2] + 1.2 S[3]   0
 wheat_average : -0.7 S[4] + 0.3 S[5] + 1.3 S[6]   0
 corn_average : -0.6 S[7] + 0.4 S[8] + 1.4 S[9]   0
 total_hectares : S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9]   130
 total_kilos_soy : 2900 S[1] + 3800 S[2] + 4400 S[3]   250000
 total_kilos_wheat : 3500 S[4] + 4100 S[5] + 4200 S[6]   250000
 total_kilos_corn : 5900 S[7] + 6700 S[8] + 7900 S[9]   250000
 S[1]   0
 S[2]   0
 S[3]   0
 S[4]   0
 S[5]   0
 S[6]   0
 S[7]   0
 S[8]   0
 S[9]   0
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [6e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.7998422848e+02 Ph1: 7(28.3601); Du: 9(179.984) 0s
          7     1.1599940331e+05 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.1599940331e+05
P-D objective error :  6.2723824816e-17
HiGHS run time      :          0.00
```

```
@show value.(S);
@show objective_value(plant_model);
total = sum(value.(S))
print("Total hectares of space used: $total")
```

```
value.(S) = [13.812154696132604, 55.24861878453038, 0.0, 6.7433064173395625, 15.73438164045
objective_value(plant_model) = 115999.40331491713
Total hectares of space used: 130.0
```

–> Code Explanation: I used the same JuMP code setup that we used in lab 3 to solve this problem, while filling in the new obective and constraints. For simplicity, I set the variable to all be components of S, so that I can easily refer to them throughout the rest of the problem.

–> From this JuMP analysis, we found that the optimal allocation of ha for the three crops with varying pesticide applications is as follows:

Soy:

- $S_{S0} = 13.81$ ha
- $S_{S1} = 55.25$ ha

- $S_{S2} = 0.0$ ha

Wheat:

- $S_{W0} = 6.74$ ha
- $S_{W1} = 15.73$ ha
- $S_{W2} = 0.0$ ha

Corn:

- $S_{C0} = 26.92$ ha
- $S_{C1} = 0.0$ ha
- $S_{C2} = 11.538$ ha

–>

- Given these values above, the total expected profit for the farmer is $115999.40

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

–> To answer this problem, I will use the same code as in part 2.2, but I'll change the constraint which covers the total hectares available to match the new value:

```
plant_models = Model(HiGHS.Optimizer) # initialize model object
@variable(plant_models, S[1:9] >= 0) # non-negativity constraints
# uncomment the following lines and add the objective and constraints as needed for the model
@objective(plant_models, Max, 694S[1] + 948S[2] + 1094S[3] + 555S[4] + 757S[5] + 714S[6] + 9

@constraint(plant_models, soy_average, -0.8S[1] + 0.2S[2] + 1.2S[3]  <= 0)
@constraint(plant_models, wheat_average, -0.7S[4] + 0.3S[5] + 1.3S[6]  <= 0)
@constraint(plant_models, corn_average, -0.6S[7] + 0.4S[8] + 1.4S[9]  <= 0)

@constraint(plant_models, total_hectares, S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S

@constraint(plant_models, total_kilos_soy, 2900S[1] + 3800S[2] + 4400S[3] <= 250000)
@constraint(plant_models, total_kilos_wheat, 3500S[4] + 4100S[5] + 4200S[6] <= 250000)
@constraint(plant_models, total_kilos_corn, 5900S[7] + 6700S[8] + 7900S[9] <= 250000)
```

```
print(plant_models) # this outputs a nicely formatted summary of the model so you can check y

optimize!(plant_models)
```

```
Max 694 S[1] + 948 S[2] + 1094 S[3] + 555 S[4] + 757 S[5] + 714 S[6] + 908 S[7] + 1014 S[8] +
Subject to
 soy_average : -0.8 S[1] + 0.2 S[2] + 1.2 S[3]   0
 wheat_average : -0.7 S[4] + 0.3 S[5] + 1.3 S[6]   0
 corn_average : -0.6 S[7] + 0.4 S[8] + 1.4 S[9]   0
 total_hectares : S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9]   140
 total_kilos_soy : 2900 S[1] + 3800 S[2] + 4400 S[3]   250000
 total_kilos_wheat : 3500 S[4] + 4100 S[5] + 4200 S[6]   250000
 total_kilos_corn : 5900 S[7] + 6700 S[8] + 7900 S[9]   250000
 S[1]   0
 S[2]   0
 S[3]   0
 S[4]   0
 S[5]   0
 S[6]   0
 S[7]   0
 S[8]   0
 S[9]   0
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [6e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.7998422848e+02 Ph1: 7(28.3601); Du: 9(179.984) 0s
          7     1.2296340331e+05 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.2296340331e+05
P-D objective error :  5.9171491942e-17
```

```
HiGHS run time      :          0.00
```

```
@show value.(S);
@show objective_value(plant_models);
total = sum(value.(S))
print("Total hectares of space used: $total")
```

```
value.(S) = [13.8121546961326, 55.24861878453038, 0.0, 9.743306417339564, 22.734381640458977
objective_value(plant_models) = 122963.40331491713
Total hectares of space used: 140.0
```

–> As we can see from this new analysis, buying 10 extra hectares of land would provide an additional $6964 of profit for the farmer, assuming the optimal solutions are applied to pesticide and crop allocation.

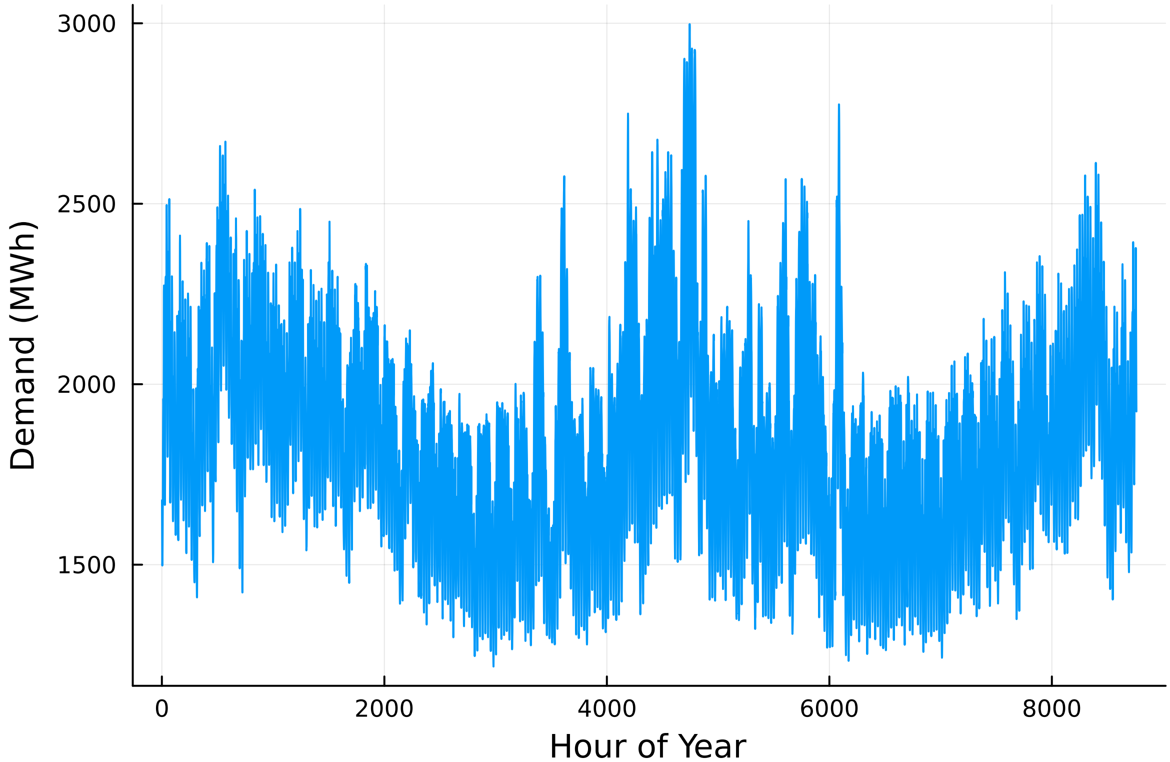
–> This means that I would only advise the farmer to adopt this plan if the cost of the 10 hectares was less than the net profit it would create for them.

–> Adding more land after having 130 ha already is not an efficient use of time, because the maximum amounts of soybeans and corn are already being produced. We know this because when increasing the max amount of hectares available, only the ha for wheat goes up. Most likely the limiting factor is the total kg limit, which the soybeans and corn have already hit at 130 ha total. Since wheat is not a very profitable crop, buying extra land to grow solely more wheat usually won't be a good economic investment.

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```julia
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

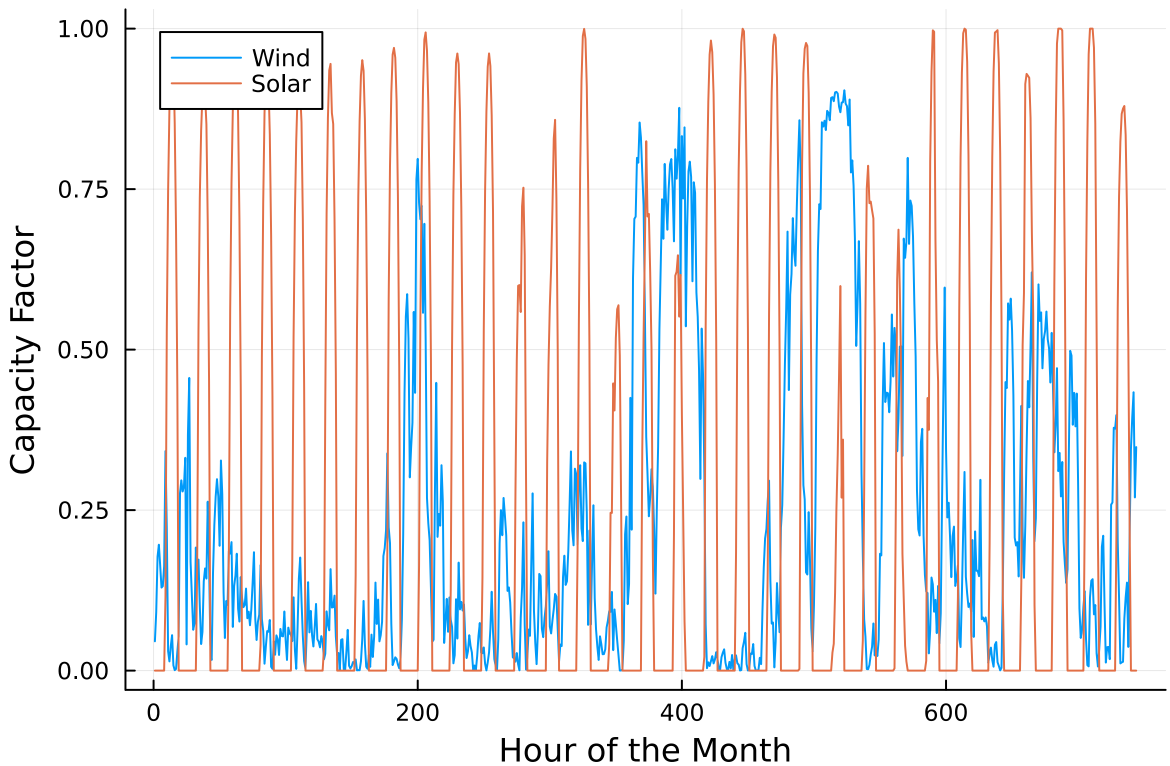
While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.

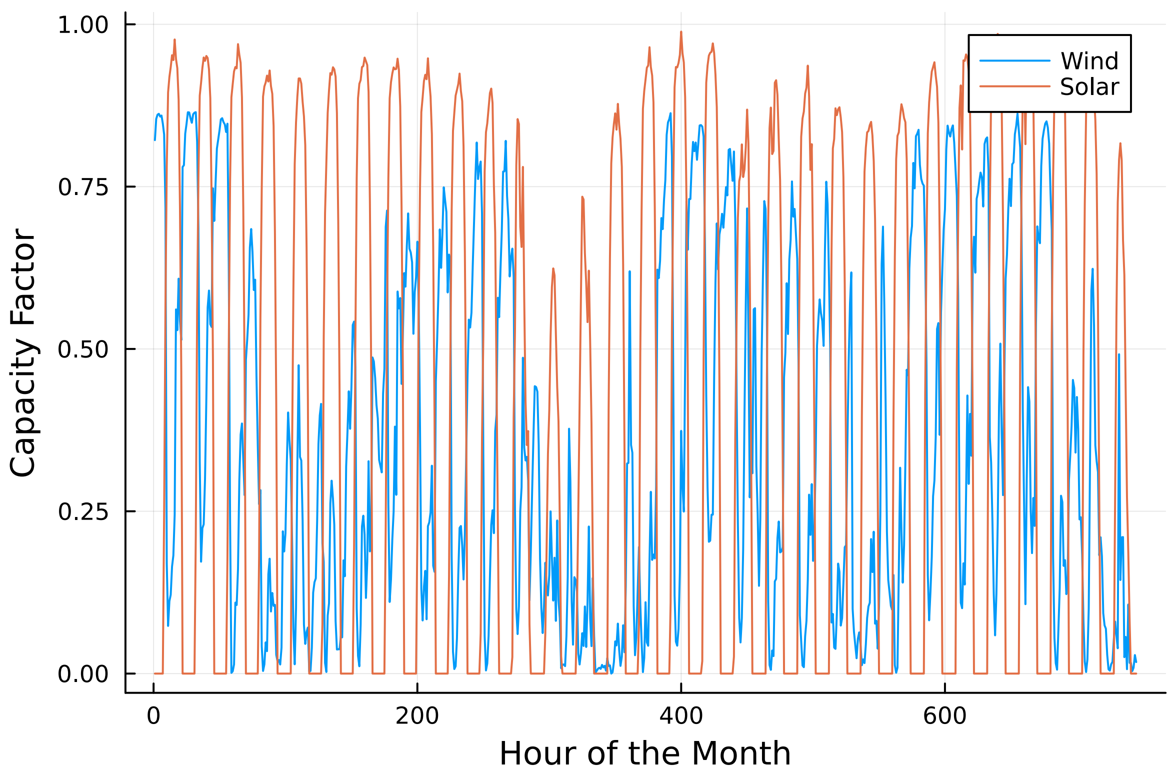
Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

> **Significant Digits**
>
> Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.
>
> **Getting Variable Output Values**
>
> `value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.
>
> **Suppressing Model Command Output**
>
> The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

References

List any external references consulted, including classmates.